PART 1 — Data Structure (Data Storage Design)

1. Introduction

This part explains **how data should be stored** in a system to make it:

- fast
- efficient
- easy to access
- easy to maintain

To do this, we must choose between **files** and **databases** as the storage format.

---

2. Data Storage Formats

Two main choices:

A. Files

A file is just a simple list of records saved in the computer.

Types of files:

- **Master file** – stores permanent, important data (e.g., customers).
- **Look-up file** – stores fixed lists (e.g., country names).
- **Transaction file** – stores temporary updates before applying them to a master file.

Pros: simple, cheap, easy
Cons: no complex searching, no relationships, not scalable

B. Databases

A database stores information in a structured way with relationships.

Types of databases:

1. Legacy databases

Old systems rarely used today:

- **Hierarchical**
- **Network**

Fast but hard to update; require too much programming.

2. Relational Databases (Most Common Today)

Use **tables**, **primary keys**, **foreign keys**.

Example: MySQL, PostgreSQL, Oracle
Use **SQL** to manage data.

3. Object-Oriented Databases (OODBMS)

Used where data is complex (images, videos).
Work with **objects**, **classes**, **subclasses**.

4. Multidimensional Databases

Used for **data warehousing** and **decision support** (DSS).
Allow slicing, dicing, and analysis.

---

3. How to Select a Storage Format

Depends on:

✔ Data type

- simple → relational
- multimedia → object
- analytics → multidimensional

✔ Type of system

- transaction processing (many updates) → relational, file, object
- decision support (read-only data) → relational, multidimensional

✔ Existing systems

Pick what the company already uses (reduces learning cost).

✔ **Future needs**

Choose technologies that are widely supported.

---

4. Moving from Logical to Physical Data Models

The logical ERD (business view) must be converted to a physical ERD (implementation view).

Steps:

1. Entities → Tables/Files
2. Attributes → Fields
3. Add Primary Keys
4. Add Foreign Keys
5. Add system-related details (indexes, audit fields, etc.)

PART 2 — Algorithm (Program Design)

1. What Is an Algorithm?

A step-by-step procedure for solving a problem.
In program design, algorithms describe **how a system should behave**.

---

2. Moving from Logical to Physical Process Models

We convert the logical DFD into a physical DFD by:

- **Add Implementation Details**
  - Add technical information such as file names, database tables, validation steps, and system methods not shown in the logical DFD.
- **Draw the Human–Machine Boundary**
  - Separate **manual tasks** done by humans from **automated processes** performed by the computer system.

- **Add System-Related Processes**
    - Include technical processes like error handling, logging, backups, authentication, and other system-level tasks.
- **Update Data Flows**
    - Show more detailed data, including specific fields, formats, system messages, and technical elements added for implementation.
- **Update Metadata in the CASE Repository**
    - Record all technical details (table names, field types, data sizes, timing, etc.) so developers know how to implement the system.

---

3. Designing Programs

Programs must be:

- modular
- clear
- easy to update

Tools used:

Structure Chart

A **structure chart** is a graphical tool used in system design to show how a program is organized. It breaks a large program into smaller, manageable modules and shows how those modules interact. Structure charts are mostly used in **top-down modular design**, where a system is divided step-by-step into simpler parts.

Purpose of a Structure Chart

The main purpose is to:

- show **the overall organization** of a program
- break the system into **independent modules**
- make the program easy to understand, develop, test, and maintain

It helps programmers see **which module calls which**, and **what data is passed** between them.

Components of a Structure Chart

1. Modules

A module is a small unit of code that performs **one specific function**.
Modules are drawn as rectangles.

Types of modules:

- **Control Module** – manages and calls other modules
- **Subordinate Module** – performs detailed tasks when called

---

2. Connections

Lines connect modules and show the flow of control.

- **Control links** – show which module calls another
- **Data couples** – show data being passed between modules
- **Control couples** – show system messages or parameters
  (arrows with filled circles represent control messages)

---

3. Sequence, Selection, and Iteration

Structure charts can show:

- **Sequence** → modules executed in order
- **Selection** → modules executed based on a condition (if/else)
- **Iteration** → modules repeated in loops

These make the chart represent real logical behavior of the program.

Building a Structure Chart — Steps

1. **Identify Modules**
   Convert DFD processes into program modules.
2. **Organize the Modules**
   Arrange them hierarchically from general → specific.
3. **Add Loops and Decisions**
   Show which modules repeat or run conditionally.
4. **Add Data and Control Couples**
   Identify the exact data or messages passed between modules.
5. **Review for Quality**
   Check cohesion (single purpose) and coupling (independence).

Design Rules:

- High cohesion (each module does one thing)
- Low coupling (modules are independent)
- High fan-in (module reused by many others)
- Avoid high fan-out (too many submodules under one module)

⭑ 4. Program Specification

A detailed guide for programmers including:

1. Program information (name, developer, deadline)
2. Events that trigger the program
3. Inputs and outputs
4. Pseudocode (step-by-step instructions)

This ensures programmers know exactly what to build.
